# Gravitational Wave Open Data Workshop

## Tutorial 2.1: Generating Waveforms

This tutorial shows how to numerically obtain the gravitational waveform radiated during a compact binary coalescence, assuming the basic parameters of the binary are known.

We will be using the PyCBC library, which provides an easy-to-use Python interface to obtain such waveforms. PyCBC can more generally be used to analyze or simulate gravitational-wave data, find or simulate astrophysical signals from compact binary mergers, and study their parameters. It is one of the tools routinely used by groups within and outside of the LIGO and Virgo collaborations. See additional examples and documentation.

View this tutorial on Google Colaboratory or launch mybinder.

```
# Those 2 lines are just to avoid some harmless warnings when importing packages
import warnings
warnings.filterwarnings("ignore", "Wswiglal-redir-stdio")
```

## Installation (execute only if running on a cloud platform or if you haven't done the installation already!)

> ⚠️ **Warning**: restart the runtime after running the cell below.
>
> To do so, click "Runtime" in the menu and choose "Restart and run all".
>
> You may see error messages but installation usually works. If you experience problems, please report an issue.

```
# -- Uncomment following line if running in Google Colab
! pip install -q PyCBC==2.10.0 lalsuite==7.25 cryptography==43.0.0
```

## Initialization

```
import numpy as np
import scipy
import matplotlib.pyplot as plt

from ipywidgets import interact, interactive, fixed, interact_manual, widgets
```

## Generate your first waveform!

Here we'll generate the gravitational waveform using one of the available waveform approximants. The waveform can be generated as a time series using `get_td_waveform()`.

The key parameters are the masses of the binary (given in solar masses), the time between samples (in seconds), the starting gravitational-wave frequency (Hz) and the name of the approximant we'd like to use. A variety of approximants are available that include different physical effects.

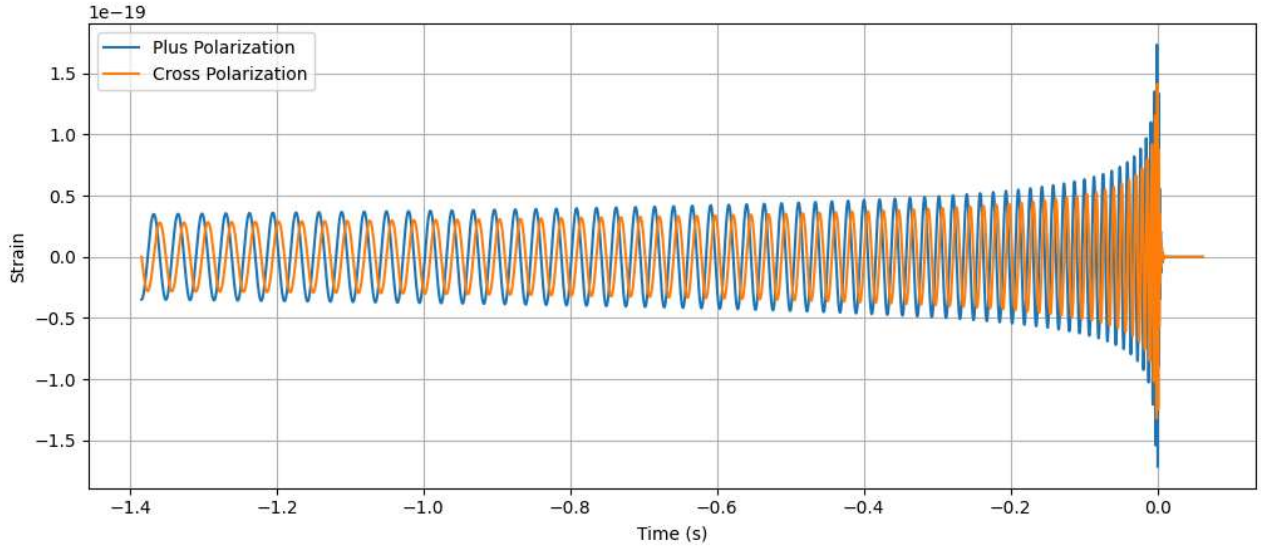
For our first example, let's use the `IMRPhenomT` approximant in the time domain (paper). This approximant assumes circular orbits with aligned spins.

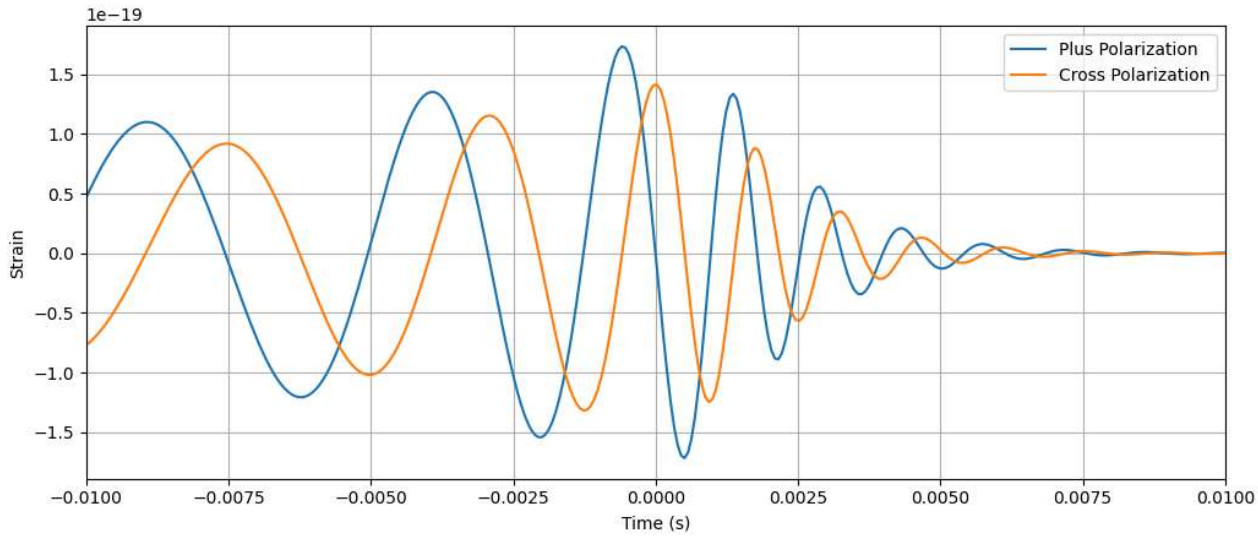
```
from pycbc.waveform import get_td_waveform

# The outputs of this function are the "plus" and "cross" polarizations
# of the gravitational-wave signal as viewed from the line of sight at
# a given source inclination
# (assumed inclination of pi/3 here, the default is zero inclination if none is provided)
hp, hc = get_td_waveform(approximant="IMRPhenomT",
                         mass1=15,
                         mass2=10,
                         inclination=np.pi/3,
                         delta_t=1.0/16384,
                         f_lower=30)

plt.figure(figsize=plt.figaspect(0.4))
plt.plot(hp.sample_times, hp, label='Plus Polarization')
plt.plot(hp.sample_times, hc, label='Cross Polarization')
plt.xlabel('Time (s)')
plt.ylabel('Strain')
plt.legend()
plt.grid()

# Zoom in near the merger time
plt.figure(figsize=plt.figaspect(0.4))
plt.plot(hp.sample_times, hp, label='Plus Polarization')
plt.plot(hp.sample_times, hc, label='Cross Polarization')
plt.xlabel('Time (s)')
plt.ylabel('Strain')
plt.xlim(-.01, .01)
plt.legend()
plt.grid()
```




Note that here we have aligned spins, so the orbital plane of the binary does not precess (i.e. the individual black holes spins are aligned with the orbital angular momentum). We also assume zero eccentricity. In the zoom-in plot, we can see the merger itself and

the ringdown that follows.

There are many other approximants available, with different methodologies employed and physical effects modelled. A full review of the models is outside the scope of this tutorial. You can get a list of all the time-domain models with `td_approximants()`.

```
# We can list the waveforms available for get_td_waveform
from pycbc.waveform import td_approximants

# List of td approximants that are available
print('Time domain waveforms: ', td_approximants())
```

```
'SEOBNRv4HM_ROM', 'SEOBNRv5_ROM', 'IMRPhenomD_NRTidal', 'SpinTaylorF2', 'TaylorF2NL', 'PreTaylorF2', 'SpinTaylorF2_SWAPPER']
```

There are some additional examples using this interface here.

## Waveforms in the frequency domain

We can also generate waveforms in the frequency domain (or Fourier domain) using the function `get_fd_waveform()`. We will talk in depth about the Fourier domain in the next tutorials.

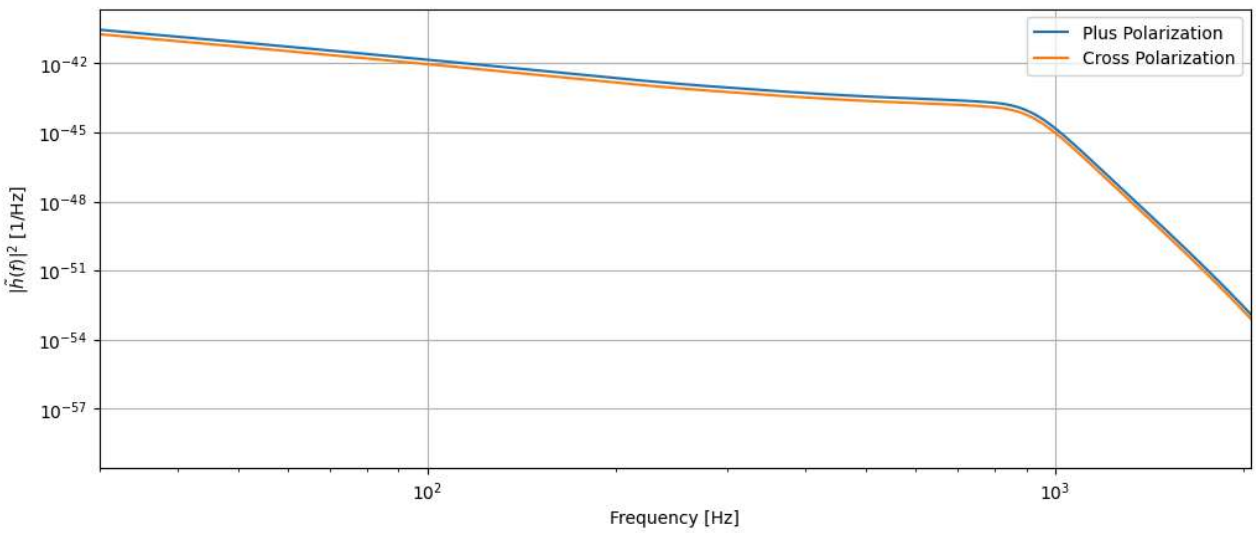
We will use the `IMRPhenomXAS` approximant in the Fourier domain (paper). As `IMRPhenomT`, it assumes circular orbits with aligned spins.

```
from pycbc.waveform import get_fd_waveform

hp, hc = get_fd_waveform(approximant="IMRPhenomXAS",
                         mass1=10,
                         mass2=10,
                         f_lower=30,
                         inclination=np.pi/3,
                         delta_f=1./8)

plt.figure(figsize=plt.figaspect(0.4))

# We will plot the norm of the Fourier components of the signal polarizations.
plt.loglog(hp.sample_frequencies, hp.squared_norm(), label='Plus Polarization')
plt.loglog(hp.sample_frequencies, hc.squared_norm(), label='Cross Polarization')
plt.xlabel('Frequency [Hz]')
plt.ylabel(r'$|\tilde{h}(f)|^2$ [1/Hz]')
plt.xlim([30,2048])
plt.legend()
plt.grid()
```




As in the time-domain, there exists many models based on different assumptions.

```
from pycbc.waveform import fd_approximants

# List of fd approximants that are currently available
print('Frequency domain waveforms: ', fd_approximants())
```

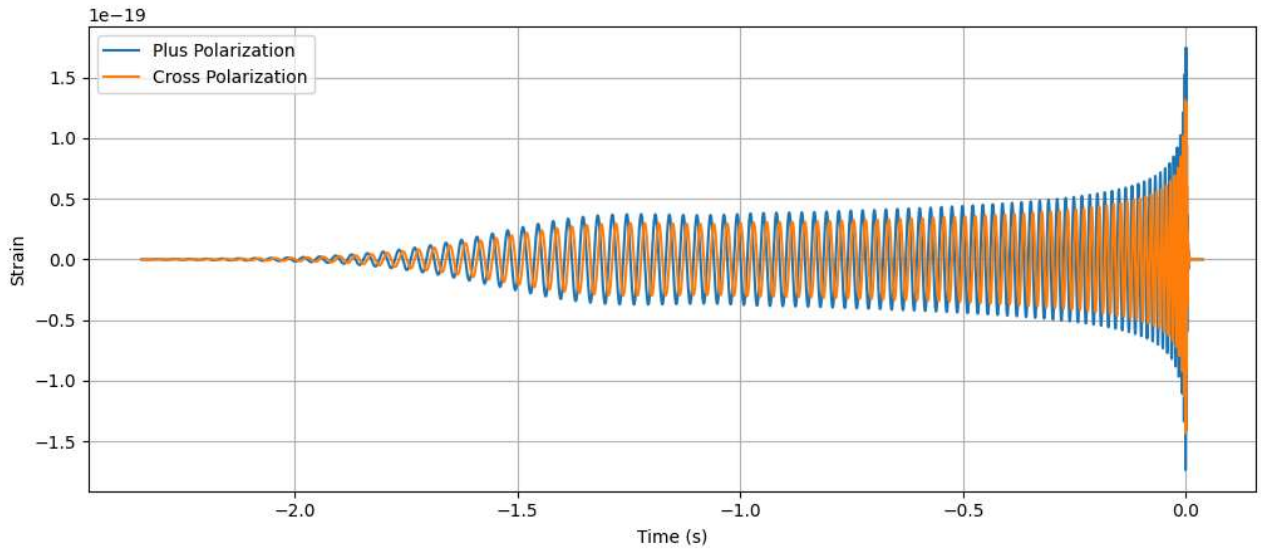
```
RPhenomA', 'IMRPhenomB', 'IMRPhenomC', 'IMRPhenomD', 'IMRPhenomD_NRTidal', 'IMRPhenomD_NRTidalv2', 'IMRPhenomNSBH', 'IMRPheno
```

## Time-domain waveforms from Fourier-domain approximants

The function `get_td_waveform()` also allows us to use a Fourier-domain approximant. Under the hood, the waveform is first generated in the Fourier domain before being inverse-Fourier transformed back to the time domain, with appropriate conditioning. This is why the amplitude is smoothly tapered at the beginning. We will discuss more on Fourier transforms and conditioning in another tutorial.

```
# The outputs of this function are the "plus" and "cross" polarizations
# of the gravitational-wave signal as viewed from the line of sight at
# a given source inclination
# (assumed inclination of pi/3 here, the default is zero inclination if none is provided)
hp, hc = get_td_waveform(approximant="IMRPhenomXAS",
                         mass1=15,
                         mass2=10,
                         inclination=np.pi/3,
                         delta_t=1.0/16384,
                         f_lower=30)

plt.figure(figsize=plt.figaspect(0.4))
plt.plot(hp.sample_times, hp, label='Plus Polarization')
plt.plot(hp.sample_times, hc, label='Cross Polarization')
plt.xlabel('Time (s)')
plt.ylabel('Strain')
plt.legend()
plt.grid()
```




## Playing with the physical parameters

In order to acquire a qualitative sense of the effect of the most important physical parameters on the waveforms, in the cell below we continuously vary the following parameters:

- the total mass $M = m_1 + m_2$ in solar masses. All masses are the 'redshifted' or 'detector-frame' masses, instead of the source-frame masses: $M = (1+z) * M_{\text{source}}$; these redshifted masses are the ones that the detector actually observes. We recall also that for binary black holes (vacuum GR), there is a universality relation: rescaling all times to $t/M$, frequencies to $f * M$ and strain to $Dh/M$, where $M$ is the total mass and $D$ is the luminosity distance, BBH waveforms become universal.
- the mass ratio $q = m_2/m_1 < 1$
- the z-component of the dimensionless spin of body 1, $\chi_1$ (we set all other spin components to 0)
- the luminosity distance to the source $D$, in Mpc; the amplitude of the observed signal taking into account the cosmological expansion is $\propto 1/D$.
- the inclination angle, in radians

For now we use `IMRPhenomT`, which allows for spins aligned with the orbital angular momentum, but does not contain higher harmonics, does not allow for spin precession or eccentricity.

All these waveforms are plotted in the same time interval, and have the same starting frequency at 30Hz.

⚠️ The cell below uses interactive plot which is not displayed correctly on GitHub but works well on Colab or when running locally.

```
def plot_waveform(M, q, chi1, dist, inc):
    mass1 = M * 1/(1+q)
    mass2 = M * q/(1+q)
    hp, hc = get_td_waveform(approximant="IMRPhenomT",
                             mass1=mass1,
                             mass2=mass2,
                             spin1z=chi1,
                             inclination=inc,
                             distance=dist,
                             delta_t=1.0/2048,
                             f_lower=30)
    fig, ax = plt.subplots(1,1, figsize=[16,4])
    ax.plot(hp.sample_times, hp, label='hp', color='C0')
    ax.plot(hc.sample_times, hc, label='hc', color='C1')
    ax.set_xlim(-1., 0.2)
    ax.set_ylim(-5e-21, 5e-21)
    ax.legend(loc='upper right')
    ax.set_xlabel('t (s)')
    ax.grid()

interact(
    plot_waveform,
    M = widgets.FloatSlider(value=30, min=1, max=100, step=1, description='M'),
    q = widgets.FloatSlider(value=0.8, min=0.05, max=1, step=0.01, description='q'),
    chi1 = widgets.FloatSlider(value=0., min=-1., max=1., step=0.01, description='chi1'),
    dist = widgets.FloatSlider(value=100, min=40, max=400, step=5, description='Dist.'),
    inc = widgets.FloatSlider(value=np.pi/3, min=0., max=np.pi, step=0.02, description='Inc.')
)
```

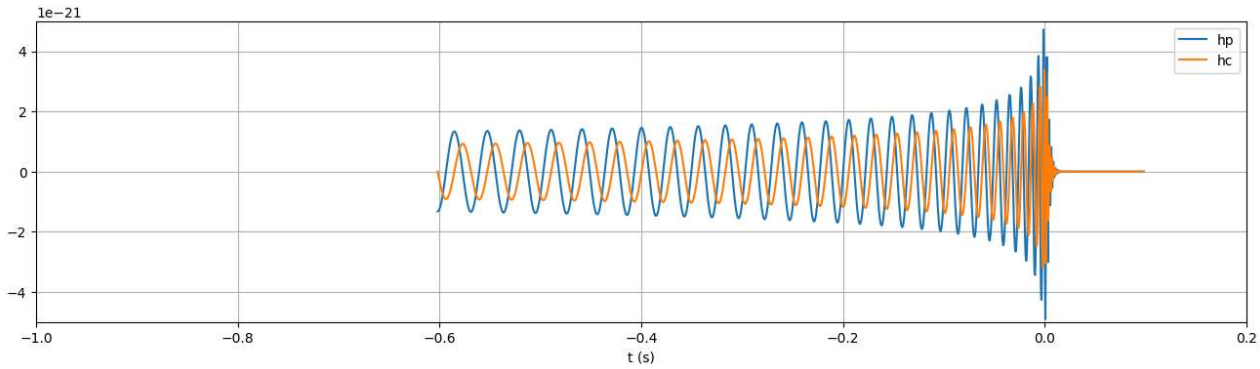
M 40.00

q 1.00

chi1 0.36

Dist. 55.00

Inc. 1.16




**plot_waveform**
`def plot_waveform(M, q, chi1, dist, inc)`

`<no docstring>`

Take some time get a feeling of the effect of some parameters in particular the mass and the distance.

## Going further

If you have time, you might have a look at the `Extracurricular_topics` notebook in this folder.

## Quiz Questions

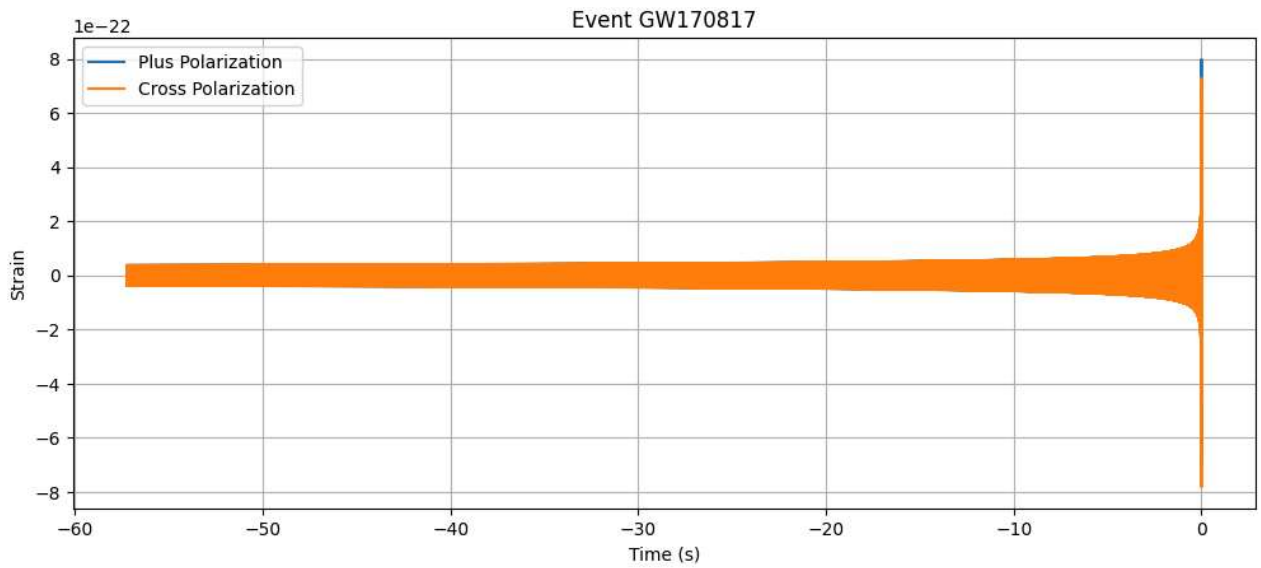
We will plot the waveforms associated to some real events. Look up the GWTC-1 catalog page to obtain the estimated masses and distance for these events.

### Question 1

Generate and plot the time-domain waveform associated to the binary neutron star merger GW170817. How much time does the signal last using a lower frequency of 30 Hz?

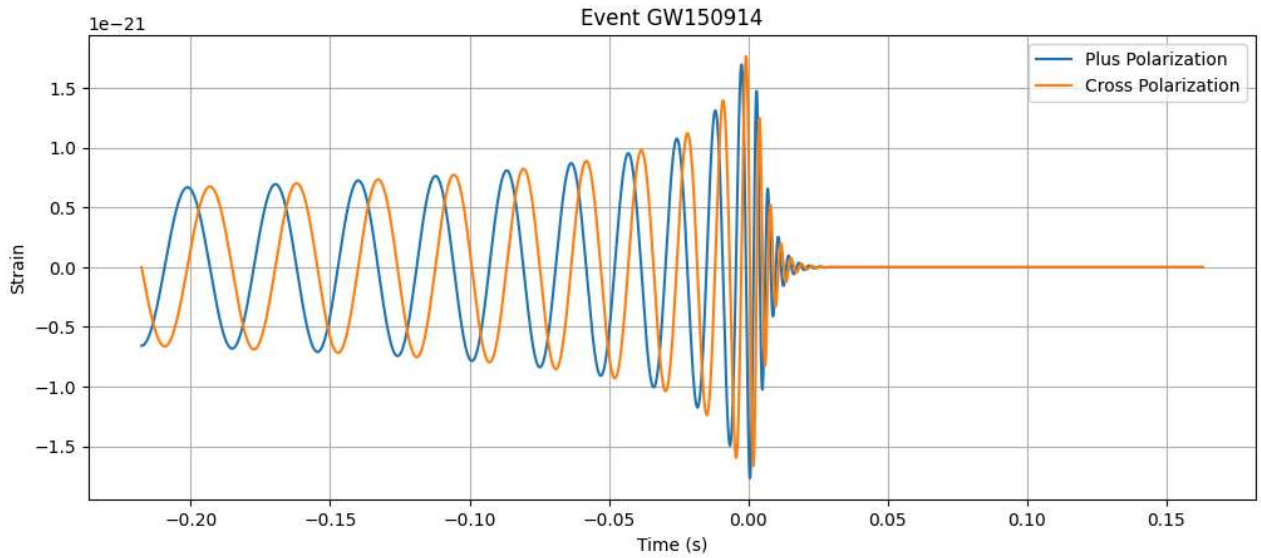
```
from pycbc.waveform import get_td_waveform
import matplotlib.pyplot as plt
approximant = 'IMRPhenomT'
# Parameters obtained from GWTC-1
# Masses in solar mass units
# Distance in Megaparsec
GW170817_m1 = 1.46
GW170817_m2 = 1.27
GW170817_d  = 40
GW170817_hp, GW170817_hc = get_td_waveform(
    approximant=approximant,
    mass1 = GW170817_m1,
    mass2 = GW170817_m2,
    distance = GW170817_d,
    delta_t = 1.0/16384,
    f_lower = 30
)

plt.figure(figsize=plt.figaspect(0.4))
plt.plot(GW170817_hp.sample_times, GW170817_hp, label='Plus Polarization')
plt.plot(GW170817_hp.sample_times, GW170817_hc, label='Cross Polarization')
plt.xlabel('Time (s)')
plt.ylabel('Strain')
plt.legend()
plt.grid()
plt.title('Event GW170817')
plt.show()
```




### Question 2

Same question for the binary black hole merger GW150914.

```
from pycbc.waveform import get_td_waveform
import matplotlib.pyplot as plt
approximant = 'IMRPhenomT'
# Parameters obtained from GWTC-1
# Masses in solar mass units
# Distance in Megaparsec
GW150914_m1 = 35.6
GW150914_m2 = 30.6
```

```
GW150914_m2 = 30.6
GW150914_d  = 440
GW150914_hp, GW150914_hc = get_td_waveform(
    approximant=approximant,
    mass1 = GW150914_m1,
    mass2 = GW150914_m2,
    distance = GW150914_d,
    delta_t = 1.0/16384,
    f_lower = 30
)

plt.figure(figsize=plt.figaspect(0.4))
plt.plot(GW150914_hp.sample_times, GW150914_hp, label='Plus Polarization')
plt.plot(GW150914_hp.sample_times, GW150914_hc, label='Cross Polarization')
plt.xlabel('Time (s)')
plt.ylabel('Strain')
plt.legend()
plt.grid()
plt.title('Event GW150914')
plt.show()
```